TRABAJO FIN DE GRADO

Grado en Ingeniería Informática

UNIVERSIDAD DE ALMERIA

ESCUELA SUPERIOR DE INGENIERÍA

Desarrollo de un videojuego roguelike 2D con generación procedural en Unity

Curso: 2025/2026

Alumno/a:

Naoufal Saidi Hassani

Director/es:

Nicolás Padilla Soriano

Dedicado a mi familia, por su apoyo incondicional y su amor constante, que han sido mi fuerza motriz en cada paso de este viaje académico.

ÍNDICE GENERAL

Página

ABREVIATURAS VIII

Resumen y Abstract **IX**

1. Introducción **1**

1.1. Contexto y relevancia del problema 1

1.2. Propósito del trabajo 2

1.3. Descripción del proyecto 2

1.4. Objetivos 3

2. Estado del arte **5**

2.1. Definición técnica del subgénero *rogue-like* 5

2.2. Generación procedural de contenido en videojuegos 5

2.3. Arquitectura de videojuegos en tiempo real 5

2.4. Inteligencia artificial en videojuegos 2D 6

2.5. Herramientas y motores considerados 6

2.6. Justificación técnica de la elección de Unity y C# 6

3. Planificación temporal **7**

3.1. Fases del proyecto 7

3.1.1. Fase 1: Exploración inicial y definición de alcance (meses 1–3) 7

3.1.2. Fase 2: Diseño conceptual y arquitectura base (meses 4–6) 7

3.1.3. Fase 3: Implementación progresiva del núcleo del juego (meses 7–11) 8

3.1.4. Fase 4: Herramientas de editor y mejora de productividad (meses 10–13) 8

3.1.5. Fase 5: Refinamiento, pruebas y documentación (meses 14–15) 8

4. Metodología **9**

4.1. Tipo de investigación 9

4.2. Metodología de desarrollo de software 9

4.3. Proceso de implementación y validación 10

5. Resultados **11**

5.1. Sistema final implementado 11
5.2. Generación procedural de salas y flujo de juego 11
5.3. Sistema de combate, armas y enemigos 12
5.4. Sistema de jefes y condición de victoria 12
5.5. Estabilidad y coherencia del sistema 13
5.6. Herramientas de automatización desarrolladas 13
5.7. Limitaciones técnicas y aspectos no implementados 13

6. Conclusiones **15**
Bibliografía 16

A. Artefactos software **19**
A.1. Estructura del proyecto 19
A.2. Arquitectura del sistema 19
A.3. Diagramas del sistema 19
A.4. Planificación temporal 24
A.5. Capturas del sistema 26
A.6. Limitaciones del apéndice 27

ÍNDICE DE FIGURAS

A.1. Diagrama de clases del sistema 21
A.2. Diagrama de casos de uso del sistema 23
A.3. Diagrama de Gantt con la planificación temporal del proyecto 25
A.4. Menú principal del juego 26
A.5. Ejemplo de combate con enemigos 26

ABREVIATURAS

AI	Artificial Intelligence
ESI	Escuela Superior de Ingeniería
GPC	Generación Procedural de Contenido
IA	Inteligencia Artificial
InSo	Ingeniería del Software
PNJ	Personaje No Jugador
SBSE	Search Based Software Engineering
TFG	Trabajo Fin de Grado
UAL	Universidad de Almería

Resumen y Abstract

El presente Trabajo de Fin de Grado describe el diseño e implementación de un videojuego 2D del subgénero *rogue-like* desarrollado con Unity y C#. El sistema integra generación procedural de salas, gestión del flujo de juego, combate cuerpo a cuerpo y a distancia, inteligencia artificial básica de enemigos y una condición de victoria basada en encuentros periódicos. La arquitectura se ha construido con separación de responsabilidades entre subsistemas y uso de *ScriptableObjects* para la parametrización de datos. La validación se ha realizado mediante pruebas funcionales manuales en el editor de Unity, verificando la estabilidad del ciclo completo de juego. Se concluye que la solución es funcional, coherente y extensible, aunque quedan pendientes mejoras como audio, balance avanzado y persistencia de progreso.

Abstract. This Bachelor's thesis presents the design and implementation of a 2D *rogue-like* game developed with Unity and C#. The system includes procedural room generation, game flow management, melee and ranged combat, basic enemy AI, and a victory condition based on periodic encounters. The architecture separates responsibilities across subsystems and uses *ScriptableObjects* to parameterize data. Validation was carried out through manual functional testing in the Unity editor, confirming stability across the full gameplay loop. The result is a functional, coherent, and extensible solution, with pending improvements such as audio, advanced balancing, and persistent progression.

1 Introducción

La industria del videojuego se ha consolidado durante las últimas décadas como uno de los sectores tecnológicos de mayor crecimiento, superando en volumen de negocio a otras industrias culturales tradicionales [1]. Desde la perspectiva de la ingeniería del software, el desarrollo de videojuegos plantea desafíos técnicos relevantes relacionados con la arquitectura de sistemas en tiempo real, la implementación de algoritmos de inteligencia artificial, la generación procedural de contenido y el diseño de interfaces interactivas. Este contexto convierte el desarrollo de videojuegos en un ámbito adecuado para la realización de un Trabajo de Fin de Grado en Ingeniería Informática, al permitir la aplicación práctica de conocimientos adquiridos durante la formación académica.

1.1 Contexto y relevancia del problema

Los videojuegos pertenecientes al subgénero *rogue-like* presentan una serie de características técnicas que los convierten en un objeto de estudio de especial interés. Entre ellas destacan la generación procedural de niveles, la ausencia de persistencia del progreso entre partidas, la progresión limitada a cada ejecución del juego y una elevada rejugabilidad derivada de la variabilidad del contenido generado. Estas propiedades obligan a diseñar sistemas capaces de producir escenarios jugables coherentes y equilibrados de forma automática, sin intervención manual directa.

En este contexto se plantea el siguiente problema técnico: cómo diseñar e implementar una arquitectura modular que permita la generación automática de niveles jugables, la gestión concurrente de múltiples subsistemas —como la inteligencia artificial, la física, la interfaz de usuario o el control de estados— y la parametrización del contenido de manera escalable y mantenible.

La actualidad de este problema se pone de manifiesto en la proliferación de títulos comerciales pertenecientes al género en los últimos años, tales como Hades, Dead Cells, The Binding of Isaac o Enter the Gungeon. Desde un punto de vista académico, el análisis de estos sistemas permite abordar conceptos relacionados con la arquitectura de software, el uso de patrones de diseño, los algoritmos de generación procedural y la separación entre lógica de ejecución y datos configurables.

1.2 Propósito del trabajo

El propósito del presente Trabajo de Fin de Grado consiste en el diseño, la implementación y la documentación de un videojuego funcional perteneciente al subgénero *rogue-like*, utilizando Unity como motor gráfico [2] y C# como lenguaje de programación. El objetivo principal es demostrar la aplicabilidad de principios propios de la ingeniería del software en un contexto práctico, prestando especial atención a la modularidad del sistema, su escalabilidad y la automatización del flujo de trabajo mediante herramientas de desarrollo.

El proyecto aborda, de forma integrada, tres áreas de conocimiento fundamentales. En primer lugar, la arquitectura de software, mediante la aplicación de patrones de diseño como *Singleton*, *Observer* y *Strategy*, así como el uso de *ScriptableObjects* para separar la lógica de juego de los datos configurables en aquellos sistema donde resulta adecuado. En segundo lugar, los algoritmos y estructuras de datos, a través de la implementación de sistemas de generación procedural de salas, la gestión ponderada de enemigos y el control del flujo del juego mediante máquinas de estados. Por último, la ingeniería de herramientas, mediante el desarrollo de utilidades dentro del editor de Unity orientadas a automatizar tareas repetitivas y a reducir la probabilidad de error humano durante la creación de contenido.

1.3 Descripción del proyecto

El videojuego desarrollado, denominado Ascension, es un *rogue-like* bidimensional con vista cenital, en el que el jugador avanza a través de una secuencia de salas generadas de forma procedural, enfrentándose a enemigos hasta alcanzar la condición de victoria establecida. Dicha condición consiste en la derrota de tres jefes que aparecen de forma periódica cada cinco salas.

El sistema implementado incluye un flujo de juego completo que abarca el menú principal, la selección de clase entre tres personajes con estadísticas diferenciadas, la gestión de estados globales mediante un *GameManager* y las pantallas de victoria o derrota. Asimismo, se ha desarrollado un sistema de generación procedural de salas basado en *tilemaps*, con muros perimetrales, elementos de transición entre salas basados en *tiles* interactivos (escaleras).

El sistema de combate contempla dos tipos de armas, cuerpo a cuerpo y a distancia, con proyectiles físicos, detección de colisiones y un sistema configurable de armas iniciales que está preparado para extensión futura. La inteligencia artificial de los enemigos incluye tres comportamientos diferenciados —persecución, ataque a distancia y aproximación por saltos—, además de enemigos especiales con atributos incrementados que actúan como jefes de sala. La gestión de enemigos se realiza mediante generación ponderada por coste, limpieza automática de salas y control de interacción con escaleras en función del estado del combate.

El proyecto incorpora, además, una interfaz de usuario completa con visualización de la salud del jugador, mensajes contextuales y una tabla de puntuaciones persistente. De forma complementaria, se han desarrollado herramientas de automatización dentro del editor de Unity destinadas a la configuración automática de salas, la generación de salas, la gestión gráfica de bases de datos de enemigos y armas, y la aplicación de efectos sobre *tiles*. El

sistema no incluye gestión de audio ni un controlador genérico de objetos recolectables, al considerarse estos aspectos de menor prioridad para los objetivos académicos del trabajo.

1.4 Objetivos

El objetivo general del proyecto es desarrollar un videojuego *rogue-like* funcional que permita evidenciar la aplicación de principios de ingeniería del software en el ámbito del desarrollo de videojuegos, con especial énfasis en la arquitectura modular, la generación procedural de contenido y la automatización de flujos de trabajo.

De este objetivo general se derivan los siguientes objetivos específicos: diseñar e implementar una arquitectura modular basada en patrones de diseño que facilite la separación de responsabilidades entre subsistemas; desarrollar un algoritmo de generación procedural de salas que garantice la jugabilidad y la coherencia espacial del entorno; implementar distintos comportamientos de inteligencia artificial para enemigos y enemigos especiales con atributos incrementados que actúan como jefes de sala; crear un sistema de armas con generación procedural de estadísticas y mecánicas de combate cuerpo a cuerpo y a distancia; desarrollar herramientas de automatización dentro del editor de Unity para agilizar la creación y validación de contenido; implementar un sistema de persistencia de puntuaciones mediante *PlayerPrefs* con visualización de un ranking en el menú principal; y documentar el proceso de desarrollo, las decisiones técnicas adoptadas y las limitaciones encontradas.

El presente documento expone el análisis, el diseño y la implementación detallada de estos sistemas, así como las conclusiones derivadas del proceso de desarrollo.

IN LUMINE

2 Estado del arte

2.1 Definición técnica del subgénero *rogue-like*

El subgénero *rogue-like* se caracteriza por la generación procedural de niveles, la ausencia de persistencia entre partidas mediante mecánicas de muerte permanente y un alto grado de rejugabilidad. En este tipo de videojuegos, la estructura espacial y la disposición de los elementos varían en cada ejecución, lo que exige el diseño de algoritmos capaces de generar contenido coherente, jugable y equilibrado de forma automática.

Desde el punto de vista de la ingeniería del software, estas características introducen desafíos relevantes relacionados con el control de la complejidad, la validación del contenido generado y la parametrización de sistemas para garantizar una experiencia consistente bajo condiciones no deterministas.

2.2 Generación procedural de contenido en videojuegos

La generación procedural de contenido (GPC) constituye un conjunto de técnicas orientadas a la creación automática de datos mediante algoritmos, reduciendo la necesidad de diseño manual de niveles, escenarios o elementos de juego. En el contexto de los *rogue-like*, la GPC resulta un componente central, ya que permite ofrecer variabilidad estructural y funcional en cada partida.

Entre los enfoques más utilizados se encuentran los algoritmos basados en reglas, los sistemas deterministas con semillas pseudoaleatorias, los modelos fractales y los sistemas formales como los *L-systems*. Cada uno de estos métodos presenta diferentes compromisos entre control, complejidad y diversidad del contenido generado, lo que condiciona su aplicabilidad según los objetivos del proyecto.

2.3 Arquitectura de videojuegos en tiempo real

La arquitectura de videojuegos en tiempo real debe gestionar de forma eficiente la interacción continua entre el usuario y múltiples subsistemas, tales como la lógica de juego, la inteligencia artificial, la física y la interfaz de usuario. Para abordar esta complejidad, resulta habitual el empleo de arquitecturas basadas en componentes, donde las entidades del juego se definen como agregaciones de comportamientos reutilizables.

Este enfoque facilita la escalabilidad del sistema, promueve la separación de responsabilidades y permite la evolución incremental del software sin introducir dependencias rígidas entre subsistemas, aspectos fundamentales en proyectos con alto grado de iteración.

2.4 Inteligencia artificial en videojuegos 2D

La Inteligencia Artificial (IA) en videojuegos bidimensionales se orienta al diseño de comportamientos creíbles y coherentes para los personajes no jugables (PNJ). En entornos *rogue-like*, la IA debe adaptarse dinámicamente a configuraciones espaciales variables y a decisiones del jugador, manteniendo un equilibrio entre desafío y previsibilidad.

Las técnicas más empleadas incluyen máquinas de estados finitos, árboles de comportamiento y sistemas de decisión basados en reglas. Estos modelos permiten implementar comportamientos diferenciados, como persecución, ataque a distancia o evasión, con un coste computacional reducido y una alta capacidad de control sobre el diseño del comportamiento. En el presente proyecto se implementa una versión simplificada orientada a comportamientos básicos.

2.5 Herramientas y motores considerados

La selección del motor de desarrollo constituye una decisión técnica crítica en cualquier proyecto de videojuegos. Entre las alternativas más relevantes se encuentran Unity, Godot y Unreal Engine. Unity destaca por su enfoque multiplataforma, su arquitectura basada en componentes y la disponibilidad de un ecosistema amplio de recursos y documentación. Godot, como motor de código abierto, ofrece una elevada flexibilidad y un control detallado del motor, mientras que Unreal Engine proporciona capacidades gráficas avanzadas orientadas a producciones de gran escala.

La elección de una herramienta u otra depende de factores como la complejidad del proyecto, los requisitos técnicos, la curva de aprendizaje y el contexto académico en el que se desarrolla el trabajo.

2.6 Justificación técnica de la elección de Unity y C#

La selección de Unity como motor de desarrollo se fundamenta en su adecuación a proyectos académicos de ingeniería informática, su madurez tecnológica y su soporte para arquitecturas modulares [2]. La utilización de *ScriptableObject* permite separar la lógica del juego de los datos configurables, facilitando la experimentación y el ajuste de parámetros sin recompilación.

El lenguaje C# proporciona una sintaxis clara, tipado fuerte y soporte completo para programación orientada a objetos, lo que favorece la implementación de sistemas escalables y mantenibles. Su integración nativa con Unity permite centrar el esfuerzo de desarrollo en la lógica del sistema, reduciendo la complejidad asociada al entorno de ejecución.

3 Planificación temporal

El desarrollo del Trabajo de Fin de Grado se ha llevado a cabo a lo largo de un periodo aproximado de quince meses, compatibilizando su realización con una actividad laboral a tiempo completo. Esta circunstancia ha condicionado la dedicación semanal y ha dado lugar a una planificación flexible y progresiva, con fases de menor y mayor intensidad según la carga de trabajo y la madurez del proyecto.

El proyecto ha seguido una metodología iterativa e incremental, permitiendo redefinir el alcance inicial y adaptar los objetivos conforme se consolidaban las decisiones técnicas.

3.1 Fases del proyecto

Con el objetivo de organizar el desarrollo del Trabajo de Fin de Grado de manera estructurada y coherente, el proyecto se ha dividido en una serie de fases claramente diferenciadas. Cada una de estas fases responde a un conjunto de objetivos específicos y refleja la evolución progresiva del trabajo, desde la definición inicial del alcance hasta la validación final del sistema y la elaboración de la documentación.

3.1.1 Fase 1: Exploración inicial y definición de alcance (meses 1–3)

Durante esta fase se realizó una exploración inicial de posibles enfoques para el TFG, evaluando distintas ideas de proyecto y tecnologías. Se analizaron los requisitos generales del trabajo académico y se llevó a cabo una primera toma de contacto con el desarrollo de videojuegos en Unity.

Como resultado de esta fase se decidió redefinir el enfoque del TFG hacia el desarrollo de un videojuego del subgénero *rogue-like*, al considerarse un dominio adecuado para aplicar conocimientos de ingeniería del software.

3.1.2 Fase 2: Diseño conceptual y arquitectura base (meses 4–6)

En esta fase se definieron los objetivos generales y específicos del proyecto, así como la arquitectura inicial del sistema. Se diseñaron los principales subsistemas del juego (gestión de estados, flujo de salas, generación procedural y combate) y se establecieron las bases del proyecto en Unity.

Se tomaron decisiones clave relativas al uso de *ScriptableObjects*, patrones de diseño y separación entre lógica y datos.

3.1.3 Fase 3: Implementación progresiva del núcleo del juego (meses 7–11)

Esta fase constituyó el núcleo del trabajo práctico. Se implementaron de forma incremental los sistemas fundamentales del videojuego:

- Generación procedural de salas
- Gestión del flujo de juego
- Sistema de combate y armas
- Inteligencia artificial de enemigos
- Sistema de jefes y condición de victoria.

La implementación se realizó de manera iterativa, validando la funcionalidad del juego en cada incremento y ajustando decisiones de diseño cuando fue necesario.

3.1.4 Fase 4: Herramientas de editor y mejora de productividad (meses 10–13)

De forma paralela a la implementación del juego, se desarrollaron utilidades internas de apoyo al desarrollo dentro del editor de Unity con el objetivo de automatizar tareas repetitivas y facilitar la creación y validación de contenido.

Estas herramientas permitieron acelerar el desarrollo y reducir errores, especialmente en la configuración de salas, enemigos y armas.

3.1.5 Fase 5: Refinamiento, pruebas y documentación (meses 14–15)

En la fase final se realizó un proceso de pruebas funcionales del sistema completo, verificando la estabilidad del flujo de juego y corrigiendo errores detectados. Se llevaron a cabo ajustes de balance y mejoras en la interfaz de usuario.

Finalmente, se procedió a la redacción de la memoria académica, documentando el proceso de desarrollo, las decisiones técnicas adoptadas y las limitaciones encontradas.

4 Metodología

El desarrollo del presente Trabajo de Fin de Grado se ha basado en un enfoque iterativo e incremental, orientado a la construcción progresiva de un sistema software funcional mediante la implementación, integración y validación continua de sus distintos subsistemas [3]. Este enfoque ha permitido adaptar el alcance del proyecto conforme se consolidaban las decisiones técnicas, manteniendo en todo momento la estabilidad del sistema y la coherencia de su arquitectura general.

La elección de esta metodología resulta especialmente adecuada para el desarrollo de videojuegos, un ámbito en el que la experimentación, el ajuste de mecánicas y la validación temprana de funcionalidades son factores clave para garantizar la viabilidad del producto final y reducir el riesgo técnico.

4.1 Tipo de investigación

El trabajo se enmarca dentro de una investigación de tipo aplicado, centrada en el diseño e implementación de un producto software funcional. El objetivo principal es demostrar la aplicación práctica de conceptos propios de la ingeniería del software en el contexto del desarrollo de videojuegos, concretamente en la creación de un videojuego 2D del subgénero *rogue-like*.

No se ha llevado a cabo un estudio empírico con usuarios externos ni se han aplicado técnicas formales de recogida de datos cuantitativos, dado que el objeto de estudio es el propio sistema desarrollado y su correcto funcionamiento desde un punto de vista técnico, estructural y arquitectónico.

4.2 Metodología de desarrollo de software

Se ha adoptado una metodología de desarrollo iterativa e incremental, basada en la implementación progresiva de funcionalidades completas y su validación continua. Esta decisión se justifica por la naturaleza exploratoria del proyecto y por la necesidad de redefinir determinados aspectos del diseño conforme avanzaba el desarrollo y se identificaban limitaciones técnicas o mejoras potenciales.

El proceso de trabajo se ha estructurado en iteraciones de corta duración, cada una de las cuales ha generado un incremento funcional del sistema. En cada iteración se han llevado a cabo las siguientes actividades: diseño de un subsistema concreto, implementación de la funcionalidad asociada, validación mediante pruebas funcionales manuales en el editor de

Unity, corrección de errores detectados y ajuste del alcance de iteraciones posteriores en función de los resultados obtenidos.

No se ha aplicado una metodología ágil formal como Scrum o Kanban, dado que el proyecto se ha desarrollado de manera individual. La gestión del desarrollo se ha realizado de forma autónoma, priorizando los subsistemas críticos para el funcionamiento del videojuego y adaptando la planificación a las limitaciones técnicas identificadas durante el proceso.

4.3 Proceso de implementación y validación

La implementación del sistema se ha llevado a cabo siguiendo un proceso secuencial de definición de subsistemas, codificación, pruebas funcionales y refinamiento. Cada componente del videojuego se ha desarrollado de forma modular e independiente y posteriormente se ha integrado en el sistema global, verificando su correcto funcionamiento conjunto con los subsistemas ya existentes.

El proceso de validación se ha basado en pruebas funcionales manuales ejecutadas directamente en el editor de Unity. En cada iteración se han verificado aspectos clave del sistema, entre los que se incluyen: la generación procedural de salas con muros y puntos de transición funcionales, el comportamiento de la inteligencia artificial de enemigos con distintos patrones de actuación, la aplicación correcta de efectos de baldosas sobre las entidades, el flujo de avance entre salas, la aparición periódica de enemigos especiales con estadísticas escaladas y la activación de la condición de victoria tras la superación de un número determinado de encuentros.

No se ha implementado un conjunto formal de pruebas automatizadas ni se ha realizado un proceso de pruebas de usuario con participantes externos. La validación se ha centrado en garantizar la estabilidad funcional del sistema, la ausencia de errores críticos y la coherencia general del flujo de juego.

Conviene aclarar que los denominados jefes del sistema no constituyen entidades con lógica independiente, sino que se trata de enemigos normales a los que se les aplica un escalado de atributos, incluyendo vida y daño, con el objetivo de incrementar la dificultad de forma progresiva. Asimismo, los elementos descritos como puertas en la generación de salas corresponden, desde el punto de vista funcional, a escaleras que permiten la transición entre salas consecutivas.

5 Resultados

En este apartado se presentan los resultados derivados del proceso de implementación del videojuego Ascension, describiendo el estado final del sistema, las funcionalidades alcanzadas y las limitaciones técnicas identificadas tras la ejecución de pruebas funcionales.

5.1 Sistema final implementado

Se ha implementado un videojuego completo del subgénero *rogue-like* bidimensional con vista cenital, que abarca un ciclo jugable completo desde el menú principal hasta la condición de victoria o derrota. El sistema integra un conjunto cohesionado de subsistemas que colaboran para generar una experiencia de juego funcional y extensible.

La arquitectura del sistema se ha construido mediante una separación clara de responsabilidades entre componentes. El gestor global de estados (*GameManager*) coordina la transición entre escenas y mantiene la coherencia del flujo de juego. La gestión de salas (*RoomFlowController*) controla el avance entre niveles y la aparición de enemigos según criterios predefinidos. El generador de salas (*RoomGenerator*) produce de forma procedural la estructura espacial de cada nivel mediante algoritmos basados en *tilemaps*.

El proyecto se estructura en tres escenas principales: *MenuPrincipal*, *SeleccionDeClase* y *GameScene*. La escena *GameScene* constituye el núcleo del juego, donde transcurre la totalidad del flujo mediante la regeneración dinámica de salas dentro de la misma escena.

5.2 Generación procedural de salas y flujo de juego

El flujo de juego avanza mediante un sistema de salas lineales controlado por un contador interno. El jugador accede a la siguiente sala mediante la interacción con escaleras de transición, que se activan una vez que la sala actual ha sido limpiada de enemigos. Cada nueva sala incrementa dicho contador, el cual determina la aparición periódica de encuentros especiales cada cinco salas.

La generación procedural se ha validado verificando que las salas generadas presentan muros de contención íntegros, un área jugable accesible sin obstáculos, puntos de transición correctamente posicionados y que la regeneración del suelo no interfiere con las mecánicas de colisión de enemigos y jugador.

5.3 Sistema de combate, armas y enemigos

Se ha implementado un sistema de combate que distingue entre ataques cuerpo a cuerpo y ataques a distancia. Las armas cuerpo a cuerpo realizan un escaneo de área alrededor del jugador en la dirección de orientación, aplicando daño a todos los enemigos presentes dentro del rango de ataque. Las armas a distancia generan proyectiles que se desplazan en línea recta y aplican daño al entrar en colisión con enemigos.

El juego incluye tres tipos de enemigos base con comportamientos diferenciados: enemigos perseguidores que se desplazan directamente hacia el jugador, enemigos agresivos que atacan cuando se encuentran a una distancia reducida, y enemigos de rango que mantienen la distancia y realizan ataques a distancia. Cada tipo de enemigo se configura mediante datos almacenados en *ScriptableObject*, lo que permite parametrizar atributos como velocidad, daño, vida y radio de visión sin modificar la lógica de ejecución.

La aparición de enemigos se gestiona mediante un sistema de generación ponderada por coste (*EnemyManager*), que instancia enemigos de forma aleatoria dentro de un radio máximo de seguridad respecto al jugador. Este mecanismo evita la aparición de enemigos en la proximidad inmediata del jugador, mejorando la experiencia de juego.

El sistema de combate se ha validado verificando que los ataques cuerpo a cuerpo aplican daño correctamente a los enemigos dentro del rango de escaneo, que los proyectiles siguen la trayectoria calculada y aplican daño al impactar, que los enemigos se desplazan conforme a su comportamiento definido, y que la eliminación de enemigos se refleja correctamente en el control de salas limpias.

5.4 Sistema de jefes y condición de victoria

Se ha implementado un subsistema de enemigos especiales que aparecen de forma periódica cada cinco salas completadas. Estos enemigos no introducen nuevas mecánicas de inteligencia artificial respecto a los enemigos base, sino que consisten en variantes escaladas de los enemigos normales, con incrementos en atributos como vida total y daño infligido.

La aparición de estos enemigos se produce en posiciones alejadas del jugador, calculadas mediante puntos predefinidos dentro de la sala, con el objetivo de garantizar una entrada progresiva al combate y evitar situaciones de daño inmediato injustificado.

La condición de victoria se define como la superación de tres encuentros de este tipo. Cada enemigo especial derrotado incrementa un contador interno que, al alcanzar dicho valor, desencadena el evento de victoria del juego. Al cumplirse esta condición se muestra la pantalla de victoria, desde la cual es posible retornar al menú principal o iniciar una nueva partida.

La validación de este subsistema ha confirmado que el contador de salas controla correctamente la aparición de estos encuentros, que las estadísticas de los enemigos especiales se escalan adecuadamente respecto a los enemigos base y que el evento de victoria se activa conforme a lo previsto.

5.5 Estabilidad y coherencia del sistema

Tras la ejecución de múltiples sesiones de pruebas funcionales se ha validado la estabilidad del flujo completo de juego. Se ha verificado que la transición entre escenas no genera errores de referencia nula, que el *GameManager* mantiene la coherencia del estado global durante toda la ejecución, que la generación procedural de salas no produce inconsistencias geométricas, que los enemigos se destruyen correctamente al cambiar de sala y que la persistencia de puntuación mediante *PlayerPrefs* funciona sin corrupción de datos.

El sistema es capaz de completar múltiples ciclos completos de juego, desde el menú principal hasta la condición de victoria, sin presentar fallos críticos ni pérdida de información. La interfaz de usuario responde adecuadamente a los cambios de estado y muestra información actualizada del jugador durante todo el flujo de juego.

No se han implementado pruebas automatizadas con cobertura estadística, por lo que la validación se ha centrado en la detección de errores durante ejecuciones manuales en el editor de Unity.

5.6 Herramientas de automatización desarrolladas

De forma paralela al desarrollo del videojuego, se han implementado herramientas específicas dentro del editor de Unity para automatizar tareas de configuración y validación. Estas herramientas incluyen un generador automático de estructuras de escena con posicionamiento de elementos de interfaz de usuario, un editor visual de parámetros de enemigos con validaciones en tiempo de edición, un sistema de aplicación masiva de efectos de baldosas mediante operaciones por lotes, y utilidades de depuración que visualizan información de estado durante la ejecución.

El uso de estas herramientas ha reducido notablemente la incidencia de errores de configuración y ha permitido iterar de forma más ágil sobre parámetros de balance sin necesidad de recompilar código.

5.7 Limitaciones técnicas y aspectos no implementados

Se han identificado las siguientes limitaciones técnicas y aspectos no implementados por restricciones de tiempo y alcance:

- El sistema de recogida de armas no integra completamente la lógica de equipamiento automático en el controlador del jugador (*PlayerController*), requiriendo integración adicional para su funcionamiento completo
- No se ha implementado un sistema integral de audio, incluyendo gestión de música de fondo, efectos de sonido y transiciones según el estado del juego
- El sistema de caídas de objetos se limita a armas y efectos de baldosas, sin incluir un sistema genérico de probabilidades para la generación de consumibles u otros tipos de ítems

- La inteligencia artificial de enemigos implementa comportamientos básicos de persecución y ataque, sin incorporar tácticas avanzadas como coordinación entre enemigos o patrones complejos de ataque
- No se ha realizado una fase exhaustiva de balance de progresión de armas, pesos de aparición de enemigos y escalado de dificultad. Los parámetros actuales permiten completar el juego, pero un proceso de balance con pruebas de usuario habría mejorado la experiencia final
- No se ha implementado un sistema de persistencia de progreso entre partidas, salvo la puntuación, por lo que cada ejecución comienza desde un estado inicial sin desbloqueables ni mejoras permanentes
- Finalmente, la documentación interna del código se ha realizado de forma parcial mediante comentarios en métodos críticos, sin alcanzar una cobertura exhaustiva de todas las clases y métodos del sistema.

En conjunto, los resultados obtenidos confirman que el sistema desarrollado cumple los objetivos técnicos establecidos para el proyecto, proporcionando una base funcional, coherente y extensible para futuras mejoras.

Los resultados obtenidos han permitido validar el correcto funcionamiento del sistema en condiciones normales de ejecución. No obstante, algunas funcionalidades previstas, como la incorporación de música, un sistema avanzado de generación de armas raras y un proceso exhaustivo de balanceo, no han sido implementadas por limitaciones temporales. Estas carencias no comprometen el núcleo funcional del sistema ni la consecución de los objetivos técnicos planteados.

6 Conclusiones

El presente Trabajo de Fin de Grado ha abordado el diseño, la implementación y la documentación de un videojuego perteneciente al subgénero *rogue-like*, desarrollado mediante el motor Unity y el lenguaje C#. El objetivo principal ha sido aplicar de manera práctica principios de ingeniería del software en un contexto real de desarrollo de videojuegos, prestando especial atención a la modularidad del sistema, la generación procedural de contenido y la automatización de procesos dentro del entorno de desarrollo.

Los resultados obtenidos permiten afirmar que se ha logrado construir un sistema software funcional y completo, capaz de soportar un ciclo de juego íntegro desde el menú inicial hasta las condiciones de victoria o derrota. La arquitectura implementada ha favorecido una clara separación de responsabilidades entre subsistemas —gestión de estados, generación de salas, combate, enemigos e interfaz de usuario—, contribuyendo a la mantenibilidad y extensibilidad del proyecto.

La generación procedural de salas ha demostrado ser una solución eficaz para garantizar la rejugabilidad y, al mismo tiempo, un elemento relevante desde el punto de vista de la ingeniería del software, al requerir algoritmos capaces de asegurar coherencia espacial y estabilidad del sistema. Asimismo, la implementación de enemigos con comportamientos diferenciados y la utilización de enemigos especiales basados en escalado de atributos ha permitido introducir progresión de dificultad sin aumentar de forma significativa la complejidad del sistema.

El desarrollo de herramientas específicas integradas en el editor de Unity ha tenido un impacto positivo en la productividad y en la reducción de errores de configuración, evidenciando la importancia del diseño de herramientas auxiliares como parte del proceso de desarrollo de software interactivo.

El proyecto presenta limitaciones derivadas principalmente del alcance temporal y de su desarrollo individual. Entre ellas se incluyen la ausencia de un sistema completo de audio, la falta de persistencia avanzada de progreso entre partidas, la implementación de inteligencia artificial basada en comportamientos simples y la inexistencia de pruebas automatizadas o evaluaciones con usuarios externos. Estas limitaciones no comprometen la validez del sistema desarrollado, pero sí definen líneas claras de trabajo futuro.

En conjunto, este trabajo ha permitido consolidar competencias fundamentales de la Ingeniería Informática, tales como el diseño de arquitecturas software, la implementación de sistemas interactivos en tiempo real, el uso de herramientas profesionales de desarrollo y la documentación técnica de proyectos complejos. Por todo ello, se considera que los objetivos planteados inicialmente han sido alcanzados de forma satisfactoria.

Desde el punto de vista de la práctica profesional en el ámbito de la ingeniería informática, el proyecto evidencia la viabilidad de aplicar metodologías iterativas e incrementales en el desarrollo individual de videojuegos, así como la utilidad de la automatización de tareas dentro del entorno de desarrollo para mejorar la productividad y reducir errores.

Bibliografía

[1] Newzoo, “Global games market report 2025: The free version,” 2025, comprehensive analysis of the global video game market, including revenue projections and demographic trends.

[2] U. Technologies, *Unity Manual 2023.3 LTS*. Unity Documentation, 2024, current industry standard for 2025 cross-platform deployment.

[3] K. Schwaber and J. Sutherland, “The scrum guide: The definitive guide to scrum: The rules of the game,” 2020, standard agile methodology for 2024-2025 game development cycles. [Online]. Available: https://scrumguides.org/scrum-guide.html

IN LUMINE

A Artefactos software

Este apéndice recoge los principales artefactos software generados durante el desarrollo del Trabajo de Fin de Grado. Su inclusión tiene como objetivo complementar la memoria principal sin sobrecargar el cuerpo del documento con detalles técnicos excesivamente específicos.

A.1 Estructura del proyecto

El proyecto ha sido desarrollado utilizando el motor de videojuegos Unity, siguiendo una organización modular basada en la separación de responsabilidades. La estructura general del proyecto se divide en los siguientes bloques principales:

- **Scripts**: contiene la lógica principal del juego implementada en C#.
- **Prefabs**: almacena los objetos reutilizables del juego, como enemigos, salas y elementos interactivos.
- **Scenes**: incluye las distintas escenas que conforman el flujo del juego.
- **Assets gráficos**: recursos visuales utilizados en el desarrollo.

Esta organización facilita el mantenimiento del código y la escalabilidad del proyecto.

A.2 Arquitectura del sistema

La arquitectura del videojuego se basa en un enfoque modular, donde cada subsistema se implementa de forma independiente. Entre los principales subsistemas destacan la gestión del jugador, la generación procedural de niveles, el control de enemigos y la gestión del flujo de juego.

Con el objetivo de aportar una visión estructural del sistema, en el siguiente apartado se incluyen los diagramas UML utilizados durante el diseño.

A.3 Diagramas del sistema

La Figura A.1 muestra el diagrama de clases del sistema, en el que se representan las principales entidades del videojuego y sus relaciones. Este diagrama permite comprender

la organización interna del código y la separación de responsabilidades entre los distintos componentes.

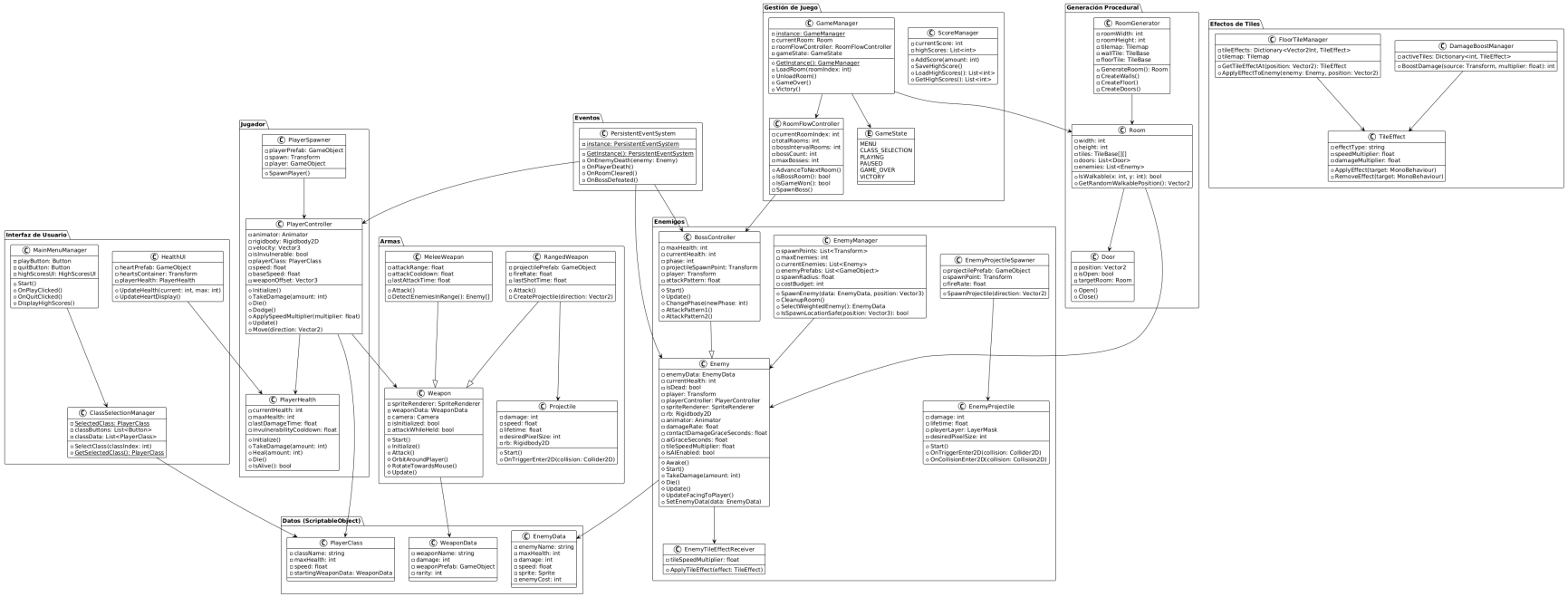

Figura A.1: Diagrama de clases del sistema

Por otro lado, la Figura A.2 presenta el diagrama de casos de uso, que describe las interacciones principales entre el jugador y el sistema, proporcionando una visión funcional del comportamiento esperado del videojuego.

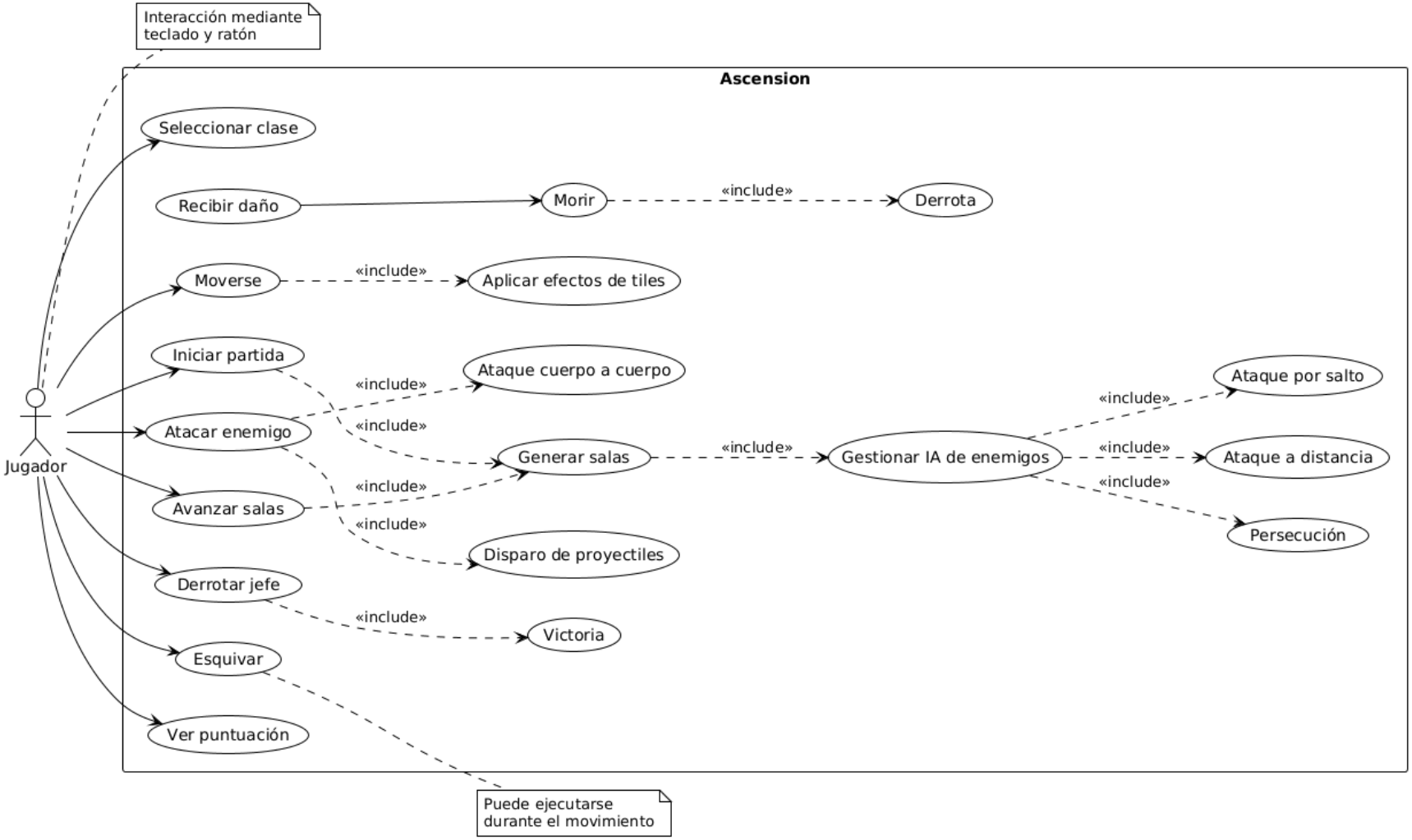

Figura A.2: Diagrama de casos de uso del sistema

A.4 Planificación temporal

La planificación del desarrollo del proyecto se ha representado mediante un diagrama de Gantt, que permite visualizar de forma clara las distintas fases del trabajo y su distribución temporal. La Figura A.3 recoge dicha planificación, mostrando la secuencia de tareas y su duración aproximada.

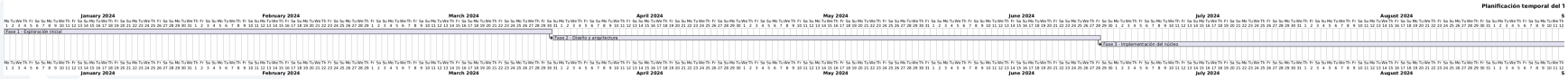

Figura A.3: Diagrama de Gantt con la planificación temporal del proyecto

A.5 Capturas del sistema

Con el objetivo de ilustrar visualmente el resultado final del sistema desarrollado, se incluyen a continuación algunas capturas representativas del videojuego en ejecución.

La Figura A.4 muestra el menú principal del juego, desde el cual el jugador puede iniciar una nueva partida y acceder a las opciones básicas del sistema.

Figura A.4: Menú principal del juego

En la Figura A.5 se observa una situación de combate, en la que el jugador interactúa con enemigos generados proceduralmente, poniendo en funcionamiento las mecánicas principales del sistema.

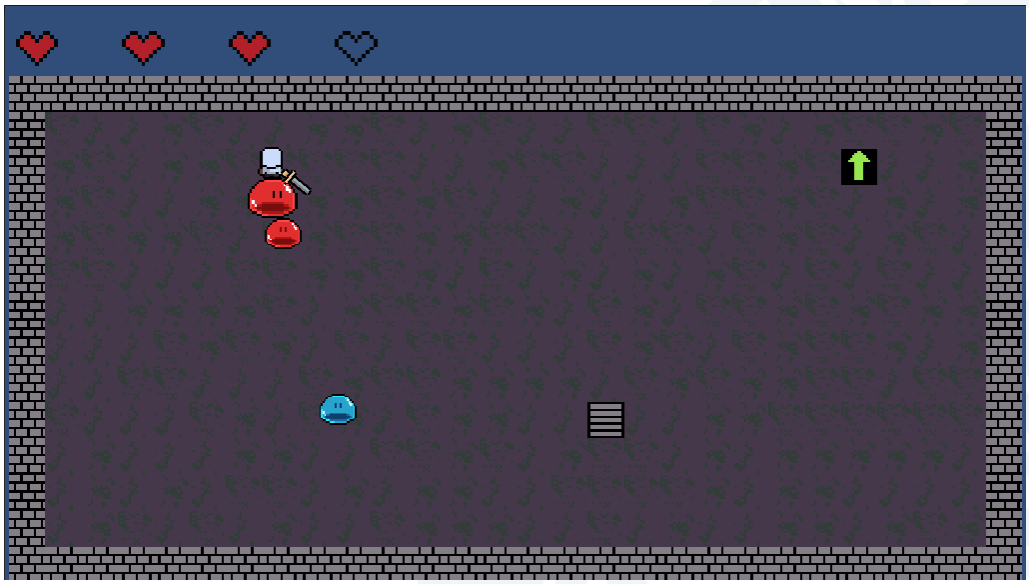

Figura A.5: Ejemplo de combate con enemigos

A.6 Limitaciones del apéndice

Por motivos de claridad y extensión, no se incluyen en este apéndice todos los scripts ni recursos del proyecto. Elementos como configuraciones detalladas, modelos completos o documentación interna del código pueden consultarse directamente en el repositorio del proyecto.

IN LUMINE

UNIVERSIDAD DE ALMERÍA

Resumen/Abstract

El presente Trabajo de Fin de Grado describe el diseño e implementación de un videojuego 2D del subgénero *rogue-like* desarrollado con Unity y C#. El sistema integra generación procedural de salas, gestión del flujo de juego, combate cuerpo a cuerpo y a distancia, inteligencia artificial básica de enemigos y una condición de victoria basada en encuentros periódicos. La arquitectura se ha construido con separación de responsabilidades entre subsistemas y uso de *ScriptableObjects* para la parametrización de datos. La validación se ha realizado mediante pruebas funcionales manuales en el editor de Unity, verificando la estabilidad del ciclo completo de juego. Se concluye que la solución es funcional, coherente y extensible, aunque quedan pendientes mejoras como audio, balance avanzado y persistencia de progreso.

Abstract. This Bachelor's thesis presents the design and implementation of a 2D *rogue-like* game developed with Unity and C#. The system includes procedural room generation, game flow management, melee and ranged combat, basic enemy AI, and a victory condition based on periodic encounters. The architecture separates responsibilities across subsystems and uses *ScriptableObjects* to parameterize data. Validation was carried out through manual functional testing in the Unity editor, confirming stability across the full gameplay loop. The result is a functional, coherent, and extensible solution, with pending improvements such as audio, advanced balancing, and persistent progression.

GRADO EN INGENIERÍA INFORMÁTICA

2025/2026